

REPORTES EJECUTIVOS - CICLO DE VIDA DEL PROYECTO

PlaneacionPUA

Fecha de generacion: 2026-03-26

Repositorio: OskarAPP/PlaneacionPUA

INFORME 1: INICIACION Y PLANEACION

ORIGEN DEL PROYECTO Y PROBLEMA TECNICO A RESOLVER

- Necesidad institucional de digitalizar y centralizar la planeacion academica universitaria, eliminando procesos manuales basados en documentos fisicos o hojas de calculo desconectadas.
- Problema core: inexistencia de un sistema unificado para gestionar el ciclo completo del documento PUA (Planeacion Universitaria Academica), incluyendo asignacion de docentes, materias, competencias y generacion de versiones en PDF.
- Requerimiento de control de acceso por roles: administradores, docentes y personal administrativo deben operar sobre vistas y datos diferenciados.
- Necesidad de importacion masiva de bibliografia desde archivos CSV/Excel para reducir carga operativa del area academica.
- Requerimiento de trazabilidad de versiones de documentos PUA para auditoria y control de cambios.

REQUERIMIENTOS CORE

- CRUD completo de entidades academicas: Facultades, Academias, Carreras, Materias, Docentes, Areas, Nucleos, Tipos de Materia, Cargos.
- Gestion de competencias genericas y especificas por carrera y facultad.
- Creacion, edicion y versionado de documentos PUA por modulos/secciones.
- Generacion y descarga de documentos PUA en formato PDF.
- Autenticacion de usuarios con roles diferenciados (Rol-Based Access Control).
- Importacion masiva de bibliografia via archivo CSV/Excel.
- Gestion de imagenes de perfil de docentes.
- Sistema de notificaciones interno.
- Despliegue en entorno Linux usando contenedores Docker.

ARQUITECTURA INICIAL PROPUESTA Y DECISIONES DE DISEÑO

- Arquitectura: API REST (backend) + SPA (Single Page Application, frontend). Separacion total entre capas de presentacion y logica de negocio.
- Backend expone una API RESTful en PHP con Laravel; el frontend la consume via fetch/axios desde React.
- Base de datos relacional normalizada (MySQL 8.0) con integridad referencial mediante claves foraneas.
- Versionado de documentos PUA modelado como entidad independiente (PuaVersion) vinculada a PuaDocumento, permitiendo historico completo por documento.
- Modularizacion del documento PUA en secciones (PuaModulo) identificadas por slug, facilitando guardado parcial e independiente.
- Almacenamiento de imagenes de perfil en base de datos (campo BLOB/referencia).
- Redis para cache y colas de trabajos asincrono (generacion de PDF pesado).

- Nginx como servidor web para el frontend en produccion; PHP-FPM para el backend.

STACK TECNOLÓGICO DEFINIDO

Backend:

- Lenguaje: PHP 8.2+
- Framework: Laravel 12
- ORM: Eloquent
- Gestor dependencias: Composer
- PDF: barryvdh/laravel-dompdf v3.1
- Hojas de calculo: phpooffice/phpspreadsheet v5.2
- Testing: PHPUnit v11.5.3
- Linter: Laravel Pint
- Autenticacion: Custom (Acceso + Rol models), compatible con Sanctum

Frontend:

- Libreria UI: React 19.1.0
- Bundler: Vite 6.3.5
- Routing: React Router DOM v7.6.1
- Estilos: TailwindCSS 3.4.17 + @tailwindcss/forms
- Iconos: React Icons 5.5.0
- Linter: ESLint 9.27.0
- PostCSS: v8.5.3 + Autoprefixer
- Lenguaje: JavaScript ES6+ (JSX)

Infraestructura:

- Contenedores: Docker + Docker Compose
- Servidor web: Nginx Alpine (frontend), PHP-FPM (backend)
- Base de datos: MySQL 8.0 (contenedor Docker)
- Cache/Colas: Redis Alpine (contenedor Docker)
- CI/CD: Sin pipelines automatizados configurados (despliegue manual via docker-compose)
- SO objetivo: Linux (produccion)

INFORME 2: GESTION DE TAREAS Y ACCIONES

EPICAS E ISSUES CRITICOS ABORDADOS

- Epica 1 — Gestion de entidades academicas base:
 - * CRUD Facultades, Academias, Carreras, Materias, Areas, Nucleos, Tipos de Materia, Cargos.
 - * Relaciones N:M entre Docentes, Facultades, Carreras y Materias.
 - * Implementacion de 22 controladores REST y 23 modelos Eloquent.
- Epica 2 — Autenticacion y control de acceso:
 - * Login con correo y password; tokens de sesion.
 - * Roles: Admin, Docente (acceso diferenciado a vistas y datos).
 - * Panel de acceso (PanelAcceso.jsx) segun rol del usuario autenticado.
- Epica 3 — Procesamiento de documentos PUA:

- * Creacion de documento PUA con datos base (materia, docente, carrera).
- * Edicion modular por secciones (subcompetencias, bibliografia, etc.).
- * Versionado automatico al completar un ciclo de edicion.
- * Generacion de PDF con DomPDF desde plantilla Blade.
- * Descarga de versiones historicas en PDF.

- Epica 4 — Gestion de competencias:

- * Competencias genericas (catalog global) con CRUD completo.
- * Competencias especificas por carrera/facultad.
- * Vinculacion de competencias a modulos PUA.

- Epica 5 — Bibliografia:

- * CRUD de referencias bibliograficas.
- * Busqueda de bibliografia por texto.
- * Importacion masiva desde archivo CSV/Excel (PHPSpreadsheet).

- Epica 6 — Dockerizacion y despliegue:

- * Containerizacion de frontend, backend, MySQL y Redis.
- * Configuracion de docker-compose.yml con redes y volumenes persistentes.
- * Nginx configurado para SPA (rutas manejadas por React Router).
- * Entrypoint de Docker para migraciones automaticas al iniciar el backend.
- * Manual de ejecucion en VM Linux documentado.

ESTRATEGIA DE BRANCHING Y CONTROL DE VERSIONES

- Repositorio en GitHub (OskarAPP/PlaneacionPUA).
- Historial de commits reducido (2 commits identificados en la rama principal):
- * Commit "Dockerizado" (2026-02-11): configuracion de infraestructura Docker.
- * Commit "Initial plan" (2026-03-26): rama de trabajo para reportes ejecutivos.
- Rama activa: copilot/reportes-ejecutivos-ciclo-vida.
- No se identificaron ramas de feature, develop o release formalizadas.
- No existe configuracion de GitHub Actions ni pipelines CI/CD automatizados.
- Control de versiones del codigo aplicacion gestionado manualmente via git.

ACCIONES OPERATIVAS CLAVE Y RESOLUCION DE BLOQUEOS

- Bloqueo: configuracion de CORS para permitir peticiones del frontend (localhost:5173) al backend (localhost:8000). Resolucion: configuracion de middleware CORS en Laravel.
- Bloqueo: migraciones no se ejecutaban automaticamente en Docker. Resolucion: script docker-entypoint.sh ejecuta `php artisan migrate --force` al iniciar.
- Bloqueo: rutas SPA de React devuelven 404 en Nginx al recargar pagina directa. Resolucion: nginx.conf con directiva `try\_files \$uri \$uri/ /index.html`.
- Accion clave: inclusion de dump SQL inicial (pua_oficial.sql) y seeders para facilitar replicacion del entorno con datos de prueba.
- Accion clave: configuracion de phpunit.xml con SQLite in-memory para tests rapidos sin dependencia de MySQL en CI local.
- Accion clave: documentacion de ejecucion en Linux (MANUAL_EJECUCION_DOCKER_VM_LINUX) para facilitar onboarding de nuevos desarrolladores y despliegue en produccion.

INFORME 3: DESARROLLO E IMPLEMENTACION

LOGICA PRINCIPAL E INTEGRACIONES CLAVE

API REST — Backend (Laravel):

- 105+ endpoints REST organizados en routes/api.php.
- Controladores con metodos estandar (index, store, show, update, destroy) mas metodos especializados (e.g., agregarFacultad, quitarCarrera, importar, generarPdf).
- Relaciones Eloquent:
 - * Docente hasMany Materias (via DocenteMateria pivot).
 - * Docente belongsToMany Facultades y Carreras.
 - * Materia belongsTo Carrera, Facultad, Area, Nucleo, TipoMateria, Academia.
 - * PuaDocumento hasMany PuaModulo y PuaVersion.
 - * CompetenciaEspecificas belongsTo Carrera y Facultad.
- Generacion de PDF: DomPDF renderiza vistas Blade con los datos del PuaDocumento y sus modulos; el archivo resultante se almacena como PuaVersion con referencia al binario o ruta de archivo.
- Importacion de bibliografia: PHPSpreadsheet parsea archivos CSV/XLSX subidos via multipart/form-data; cada fila se mapea a un modelo Bibliografia y se persiste en batch.
- Gestion de imagenes: almacenamiento de imagenes de perfil referenciadas por acceso_id, accesibles via endpoint GET /api/perfil-imagen/{id_acceso}.

SPA Frontend (React):

- App.jsx define el enrutamiento principal con React Router DOM.
- Contexto de autenticacion propagado via React Context API (PuaContext y contextos de sesion de usuario).
- Patron de paginas con formularios de registro separados de las listas (e.g., Docentes.jsx + RegistroDocentes.jsx).
- ProcesarPua.jsx: componente principal del flujo PUA, orquesta la seleccion de materia, docente, modulos y generacion de versiones.
- SidebarProcesarPua.jsx: navegacion lateral entre secciones del documento PUA durante el proceso de edicion.
- Modales reutilizables para confirmacion de acciones criticas (eliminar, generar PDF).
- TailwindCSS con configuracion personalizada (tailwind.config.js) para paleta de colores institucional.

Base de Datos:

- 20+ tablas con relaciones definidas via claves foraneas.
- Migraciones Laravel para control de esquema versionado.
- Seeders para datos iniciales (roles, cargos, tipos de materia, nucleos).
- Dump SQL (pua_oficial.sql) incluido para restauracion rapida del entorno.
- Indices unicos compuestos en Materia (nombre, carrera_id), Academia (nombre, facultad_id), Carrera (nombre, facultad_id).

DESAFIOS TECNICOS Y SOLUCIONES IMPLEMENTADAS

- Desafio: generacion de PDFs con layout complejo desde Laravel.
Solucion: DomPDF con plantillas Blade estructuradas; estilos CSS inline para compatibilidad con el renderizador HTML-to-PDF.

- Desafio: relaciones muchos-a-muchos entre Docentes, Facultades, Carreras y Materias con datos adicionales en la tabla pivot.

Solucion: modelo pivot DocenteMateria con columnas extra; uso de withPivot() y sync()/attach()/detach() en Eloquent.

- Desafio: importacion de bibliografia con validacion de duplicados y manejo de errores por fila.

Solucion: PHPSpreadsheet con iteracion fila por fila, try-catch por registro y respuesta con conteo de exitosos/fallidos.

- Desafio: versionado de documentos PUA sin perder el historial de cambios.

Solucion: cada generacion de PDF crea una nueva PuaVersion con timestamp y referencia al documento padre; PuaDocumento mantiene estado mutable.

- Desafio: autenticacion sin Sanctum/Passport en una API consumida por SPA.

Solucion: endpoint de login retorna datos de usuario y rol; el frontend persiste la sesion en estado local (Context API).

PATRONES DE DISEÑO Y OPTIMIZACIONES

- MVC estricto en Laravel: modelos (logica de datos), controladores (logica HTTP), vistas Blade (solo para generacion de PDF).

- Repository-light pattern: logica de consultas compleja encapsulada en metodos de modelos Eloquent y scopes.

- Componentes React atomicos y reutilizables: modales, sidebars y formularios extraidos como componentes independientes en /Components.

- Lazy loading implicito via React Router (separacion de rutas por modulo).

- Cache Redis disponible para consultas frecuentes (catalogos de roles, cargos, tipos de materia), configurado pero pendiente de implementacion explicita en controladores.

- Nginx con compresion gzip y cache de assets estaticos (configurado en nginx.conf para el frontend en produccion).

- Multi-stage Docker build para el frontend: stage Node 20 para compilar con Vite, stage Nginx Alpine para servir; reduce tamano de imagen final.

INFORME 4: ESTADO ACTUAL Y CIERRE

ENTREGABLES TECNICOS COMPLETADOS Y FUNCIONALES

Backend:

- [x] 22 controladores REST plenamente implementados.
- [x] 23 modelos Eloquent con relaciones definidas.
- [x] 105+ endpoints REST documentados en routes/api.php.
- [x] Sistema de migraciones y seeders operativos.
- [x] Generacion de PDF de documentos PUA via DomPDF.
- [x] Importacion masiva de bibliografia CSV/Excel.
- [x] Gestion de imagenes de perfil de docentes.
- [x] Versionado de documentos PUA.
- [x] Dump SQL inicial (pua_oficial.sql) para restauracion de entorno.
- [x] Dockerfile para contenedor PHP-FPM con endpoint de migraciones.

Frontend:

- [x] SPA React con 45+ componentes y paginas funcionales.
- [x] Enrutamiento completo con React Router DOM.
- [x] Autenticacion y panel de acceso por rol.
- [x] CRUD visual de todas las entidades academicas.
- [x] Flujo completo de procesamiento de documentos PUA (ProcesarPua.jsx).
- [x] Visualizador de versiones PUA (PuaVersion.jsx).
- [x] Interfaz de gestion de bibliografia con importacion.
- [x] Perfil de usuario con imagen.
- [x] Diseno responsive con TailwindCSS.
- [x] Dockerfile multi-stage con Nginx Alpine para produccion.

Infraestructura:

- [x] docker-compose.yml con 4 servicios (frontend, backend, MySQL, Redis).
- [x] Redes Docker internas y volúmenes persistentes para MySQL.
- [x] nginx.conf para SPA con soporte de rutas del lado cliente.
- [x] Manual de ejecucion en Linux documentado en TXT y PDF.
- [x] pua_schema.txt con esquema DBML completo de la base de datos.

DEUDA TECNICA RESTANTE Y LIMITACIONES CONOCIDAS

Seguridad:

- Autenticacion no usa tokens JWT ni Sanctum; la sesion se gestiona en el frontend sin invalidacion de sesion en el servidor (sin logout real).
- Endpoints de API sin middleware de autenticacion verificado (ausencia de auth:sanctum o similar en routes/api.php).
- Passwords almacenados como password_hash; no se confirma uso de bcrypt en todos los flujos de creacion de usuarios.

Testing:

- Tests backend limitados a los archivos de ejemplo de Laravel (ExampleTest); sin tests unitarios ni de integracion para los controladores y modelos del negocio.
- Sin tests de frontend (ningun archivo .test.js o .spec.js identificado).
- Sin cobertura de codigo medida ni reportes de calidad automatizados.

CI/CD:

- Sin pipelines de integracion o despliegue continuo configurados.
- Despliegue es manual via docker-compose; sin rollback automatizado.
- Sin entornos de staging diferenciados del de produccion.

Funcionalidad:

- Cache Redis aprovisionado en infraestructura pero no implementado explicitamente en controladores (consultas frecuentes sin cacheo).
- Sistema de notificaciones modelado (Notificacion) pero la logica de envio y visualizacion puede estar incompleta.
- Sin paginacion server-side en listados de alto volumen (potencial problema de rendimiento con grandes volúmenes de datos).
- Sin busqueda avanzada/filtros en la mayoria de los listados.
- Sin internacionalizacion (i18n); sistema en espanol unicamente.

Documentacion:

- Sin documentacion de API (Swagger/OpenAPI no configurado).
- Sin guia de contribucion (CONTRIBUTING.md ausente).
- Cobertura de documentacion tecnica basica en README.md pero sin descripcion de cada endpoint ni modelos de datos detallados.

PROXIMOS PASOS TECNICOS RECOMENDADOS

Corto plazo (estabilizacion):

- Implementar autenticacion con Laravel Sanctum + tokens en todos los endpoints protegidos de routes/api.php.
- Agregar paginacion (paginate()) en todos los metodos index() de controladores con colecciones potencialmente grandes.
- Escribir tests de Feature para los flujos criticos: login, CRUD de PuaDocumento, generacion de PDF, importacion de bibliografia.
- Configurar GitHub Actions con workflow de: lint (Pint + ESLint), tests (PHPUnit), y build Docker.

Mediano plazo (escalabilidad):

- Implementar cacheo Redis en controladores de catalogos (roles, cargos, tipos de materia, areas, nucleos) con invalidacion en mutations.
- Agregar busqueda y filtros en listados de Docentes, Materias y Carreras usando query scopes de Eloquent.
- Generar documentacion interactiva de la API con Swagger/L5-Swagger o Scribe para facilitar integracion con clientes externos o apps moviles.
- Separar entornos: desarrollo, staging y produccion con .env diferenciados y secretos gestionados de forma segura (e.g., GitHub Secrets, Vault).

Largo plazo (mantenimiento y expansion):

- Implementar pipeline CI/CD completo: GitHub Actions → build Docker → push a registry → deploy en servidor de produccion (SSH o Kubernetes).
- Considerar migracion a JWT (tymon/jwt-auth) si se requiere integracion con aplicaciones moviles o clientes de terceros.
- Agregar modulo de reportes y estadisticas academicas (numero de PUAs por facultad, docentes sin PUA asignado, etc.) con exportacion a Excel.
- Implementar sistema de auditoria (logs de cambios) en entidades criticas como PuaDocumento y cambios de roles de usuario.
- Evaluar extraccion de la logica de generacion de PDF a un worker de cola (Redis Queue) para evitar timeouts en documentos extensos.

FIN DE REPORTES EJECUTIVOS